

# Virtual Smart Lock Security Assessment

Firmware inspection, hardcoded credentials, UART attack surface, debug disclosure, remediation, and verification in a controlled Wokwi environment.

|             |                          |
|-------------|--------------------------|
| TARGET      | Arduino Uno / ATmega328P |
| ENVIRONMENT | Wokwi for VS Code        |
| AUTHOR      | m4gicxleep               |
| EDITION     | September 2026           |

Controlled lab only. No physical device or third-party system was targeted.

# What this document is

This is the technical guide for the lab. It records the build, test method, findings, and final state. The separate Word document can be written as a narrative report for the blog, with screenshots, GIFs, and personal commentary.

The guide only describes work that was actually completed. The UART issue was confirmed by source review. A live serial exploit was not performed. The VCD file remains optional and is not used as primary evidence.

## Project outcome

- A working keypad-controlled lock was built and tested in Wokwi.
- The administrator PIN was recovered from the compiled ELF.
- An unauthenticated UART unlock path was identified in source code.
- The UART command and detailed debug banner were removed.
- The rebuilt ELF was checked to confirm that the sensitive strings were absent.
- The final keypad workflow was tested again after remediation.

## Contents

|    |                                       |
|----|---------------------------------------|
| 01 | Architecture and behavior             |
| 02 | Build and firmware artifacts          |
| 03 | Scope and assessment method           |
| 04 | VSL-001: hardcoded administrator PIN  |
| 05 | VSL-002: unauthenticated UART command |
| 06 | VSL-003: debug information disclosure |
| 07 | Remediation and final verification    |
| 08 | Evidence and report notes             |
| 09 | Reproduction and references           |

01 / SYSTEM

# Architecture and behavior

The lock is a small embedded control system. The Arduino reads a PIN from a matrix keypad, compares it with the configured administrator PIN, and drives a servo that represents the lock mechanism.

NORMAL PATH  
Keypad -> PIN buffer -> credential check -> servo -> door state

INITIAL UART PATH  
UART input -> command parser -> unlock routine -> servo

FIRMWARE REVIEW PATH  
ELF -> printable strings -> recovered PIN -> keypad validation

## Connections

| Component      | Connection                | Purpose                 |
|----------------|---------------------------|-------------------------|
| Keypad rows    | D9, D8, D7, D6            | Matrix scanning         |
| Keypad columns | D5, D4, D3, D2            | Matrix scanning         |
| Servo          | D10 PWM, 5V, GND          | Lock actuator           |
| Built-in LED   | LED_BUILTIN               | Door state indicator    |
| UART TX        | D1                        | Diagnostic output       |
| Logic analyzer | D0 to TX, D1 to servo PWM | Optional signal capture |

## State model

LOCKED    servo = 0 degrees    LED = off  
UNLOCKED   servo = 90 degrees    LED = on

Valid PIN -> unlock for 3 seconds -> lock again

The servo is a model of a latch, not a physical security mechanism. The three-second delay keeps the behavior easy to observe in the simulator.

## 02 / BUILD

# Build and firmware artifacts

## Toolchain

| Item               | Version or target         |
|--------------------|---------------------------|
| Board              | Arduino Uno / ATmega328P  |
| Arduino AVR Boards | 1.8.8                     |
| AVR GCC            | 7.3.0-atmel3.6.1-arduino7 |
| Keypad             | 3.1.1                     |
| Servo              | 1.3.0                     |
| Simulation         | Wokwi for VS Code         |

## Build command

```
arduino-cli compile --fqbn arduino:avr:uno --output-dir build .
```

## Final build result

Program storage: 5050 bytes of 32256 bytes (15%)

Dynamic memory: 431 bytes of 2048 bytes (21%)

Remaining SRAM: 1617 bytes

## Artifacts

| File                       | Role                                        |
|----------------------------|---------------------------------------------|
| virtual-smart-lock.ino     | Arduino C++ source                          |
| virtual-smart-lock.ino.elf | Linked executable with sections and symbols |
| virtual-smart-lock.ino.hex | Intel HEX loaded by Wokwi                   |
| diagram.json               | Virtual components and wiring               |
| wokwi.toml                 | Firmware and simulator configuration        |

Wokwi executes the compiled firmware. Editing the source does not change the simulation until the project is rebuilt and the simulator is restarted.

# Assessment method

## Security objective

Only a user with a valid PIN should be able to unlock the door.

## Assets

- Door lock state
- Administrator PIN
- Compiled firmware
- UART maintenance interface

## Assumed attacker access

- A copy of the ELF or HEX firmware artifact
- Enough access to review exposed diagnostic functionality
- No access to a real third-party device

## Method

| Step | Action                 | Reason                                           |
|------|------------------------|--------------------------------------------------|
| 1    | Functional baseline    | Prove valid and invalid PIN behavior             |
| 2    | Firmware string review | Find readable data in the ELF                    |
| 3    | Source review          | Trace privileged paths and UART commands         |
| 4    | Remediation            | Remove the unsafe command and verbose debug data |
| 5    | Rebuild and retest     | Confirm both security and function               |

## Out of scope

Physical flash extraction, debug-port probing, fault injection, voltage glitching, side-channel analysis, radio attacks, network testing, and attacks against real hardware were not performed.

## 04 / FINDING

# VSL-001: hardcoded administrator PIN

|           |                                                       |
|-----------|-------------------------------------------------------|
| Severity  | High                                                  |
| Component | Authentication mechanism                              |
| Status    | Open at architecture level                            |
| Evidence  | PIN recovered from the ELF and accepted by the keypad |

## What was found

The administrator PIN was stored as a plaintext constant in program memory. Printable-string extraction recovered the keypad map followed by the PIN.

```
123A456B789C*0#D7319
      ^^^^ administrator PIN
```

## Why it matters

Anyone who obtains the firmware can recover a shared credential and use it through the intended keypad path. The F() macro and flash storage reduce SRAM use, but they do not protect secrets.

## Residual risk

The Arduino Uno does not provide a strong protected storage boundary. Moving the PIN to EEPROM only changes its location. Hashing a four-digit PIN still leaves a search space of 10,000 values.

## Production direction

- Provision a unique credential for each device.
- Use protected key storage or a secure element.
- Increase credential entropy.
- Add attempt throttling and lockout behavior.
- Plan secure credential rotation.

OWASP ISTG reference: ISTG-FW-SCRT-003, Usage of Hardcoded Secrets.

## 05 / FINDING

## VSL-002: unauthenticated UART unlock command

|            |                                                  |
|------------|--------------------------------------------------|
| Severity   | High                                             |
| Component  | UART command handler                             |
| Status     | Remediated                                       |
| Validation | Source review, build, and final ELF string check |

### Initial code path

```
} else if (strcmp(serialBuffer, "unlock") == 0) {  
    unlockTemporarily("UART debug console");  
}
```

The UART branch reached the same privileged routine used after a valid keypad PIN. No PIN check or authorization decision existed in that branch.

### Impact

An exposed maintenance interface could bypass the intended authentication control and change the physical state of the lock.

### What was changed

- The unlock command was removed from the help output.
- The unlock branch was removed from the UART parser.
- The final console keeps only help and status.

### Evidence statement

The issue was confirmed through source-code review. Dynamic exploitation through the serial console was not performed. Do not claim a live UART exploit in the final report.

## 06 / FINDING

## VSL-003: debug information disclosure

|            |                        |
|------------|------------------------|
| Severity   | Low                    |
| Component  | UART startup logging   |
| Status     | Remediated             |
| Validation | Final ELF string check |

### Initial startup output

```
SmartLock Firmware v0.1.0  
DEBUG BUILD ENABLED  
Type 'help' to list debug commands  
Door status: LOCKED
```

The banner exposed a firmware version, build mode, and the existence of a diagnostic command interface. This did not unlock the device by itself, but it reduced reconnaissance effort.

### Final startup output

```
SmartLock ready
```

### Logging changes

- Version and build-mode text were removed.
- The diagnostic hint was removed.
- Debug-specific door messages were replaced with generic operational messages.

A production device should define a clear logging policy. Sensitive maintenance output should not be enabled by default on exposed interfaces.

07 / FIX

# Remediation and final verification

## Final UART handler

```
void executeSerialCommand()
{
  if (strcmp(serialBuffer, "help") == 0)
  {
    Serial.println(F("Available commands:"));
    Serial.println(F("  help"));
    Serial.println(F("  status"));
  }
  else if (strcmp(serialBuffer, "status") == 0)
  {
    Serial.println(doorLocked ? F("Door status: LOCKED")
                              : F("Door status: OPEN"));
  }
  else if (serialLength > 0)
  {
    Serial.println(F("Unknown command"));
  }

  memset(serialBuffer, 0, sizeof(serialBuffer));
  serialLength = 0;
}
```

## Static check

```
$matches = avr-strings.exe -a . uild irtual-smart-lock.ino.elf |
  Select-String 'SmartLock Firmware|DEBUG BUILD|UART debug console|^unlock$'

if ($matches) { $matches }
else { 'PASS: sensitive debug and UART command strings were not found.' }
```

The check returned PASS. Function symbols such as unlockDoor may still appear in an ELF that contains debugging symbols. They are not the removed standalone UART command.

## Regression test

- An invalid PIN remained denied.
- The valid PIN still moved the servo to 90 degrees.
- The servo returned to the locked position after three seconds.

## 08 / EVIDENCE

## Files and traceability

| File                                    | What it shows                |
|-----------------------------------------|------------------------------|
| 01-lab-overview-locked.png              | Initial locked state         |
| 02-invalid-pin.png                      | Invalid PIN denied           |
| 03-valid-pin-unlock.png                 | Valid PIN behavior           |
| 04-hardcoded-pin-firmware.png           | PIN recovered from ELF       |
| 05-unauthenticated-uart-code-review.png | Unsafe UART branch in source |
| 06-debug-information-disclosure.png     | Verbose startup banner       |
| 07-final-static-verification.png        | Sensitive strings absent     |
| 08-final-functional-retest.png          | Final functional test        |

### Final artifact hashes

virtual-smart-lock.ino.elf  
0F88969509C0E5340E91D20F380647C0982A7D2EA7E23DEA9521FDA09DA38350

virtual-smart-lock.ino.hex  
4C5B5A1B382A600D7A7DDE9D3F237933EC1D260D562A8D3F97535B40A74A4606

These hashes identify the final local build. Another toolchain version may produce different bytes even when the source behavior is unchanged.

### VCD note

The project includes a small VCD capture for future signal work. It was not decoded or used to support the current findings, so it should be described as optional material only.

# Writing the blog report

Write the main report in Word using your own voice. The screenshots carry the evidence. GIFs and memes can break up dense sections, but they should not replace the technical explanation.

## Suggested order

- Why the lock was built
- Circuit and normal behavior
- Threat model and scope
- Firmware build and artifacts
- VSL-001 and the recovered PIN
- VSL-002 and the UART code path
- VSL-003 and verbose debug output
- Changes made to the firmware
- Final validation
- Hardware and simulation limitations
- What you learned

## Keep the claims precise

| Use                              | Avoid                         |
|----------------------------------|-------------------------------|
| Identified through source review | Exploited over UART           |
| Validated in Wokwi               | Tested on physical hardware   |
| Residual architectural risk      | Fully secured PIN storage     |
| Optional VCD capture             | Decoded UART and PWM evidence |

## Short scope statement

This assessment was conducted in a fully controlled virtual environment using Wokwi. No physical device or third-party system was targeted.

**10 / REPRODUCE**

# Run the project

## Install the platform and libraries

```
arduino-cli core install arduino:avr  
arduino-cli lib install Keypad  
arduino-cli lib install Servo
```

## Compile

```
arduino-cli compile --fqbn arduino:avr:uno --output-dir build .
```

## Simulate

- Open the project directory in VS Code.
- Compile before starting the simulator.
- Run Wokwi: Start Simulation from the Command Palette.
- Use the keypad to test invalid and valid PIN behavior.
- Stop and restart the simulator after rebuilding the firmware.

## Project files

```
virtual-smart-lock/  
|-- virtual-smart-lock.ino  
|-- diagram.json  
|-- wokwi.toml  
|-- README.md  
|-- evidence/  
|-- report/  
`-- output/pdf/
```

## References

Wokwi for VS Code: <https://docs.wokwi.com/vscode/getting-started>

Wokwi project configuration: <https://docs.wokwi.com/vscode/project-config>

Wokwi diagram format: <https://docs.wokwi.com/diagram-format>

Arduino CLI: <https://arduino.github.io/arduino-cli/>

OWASP IoT Security Testing Guide: <https://owasp.org/www-project-iot-security-testing-guide/>

**m4gicxleep.github.io**